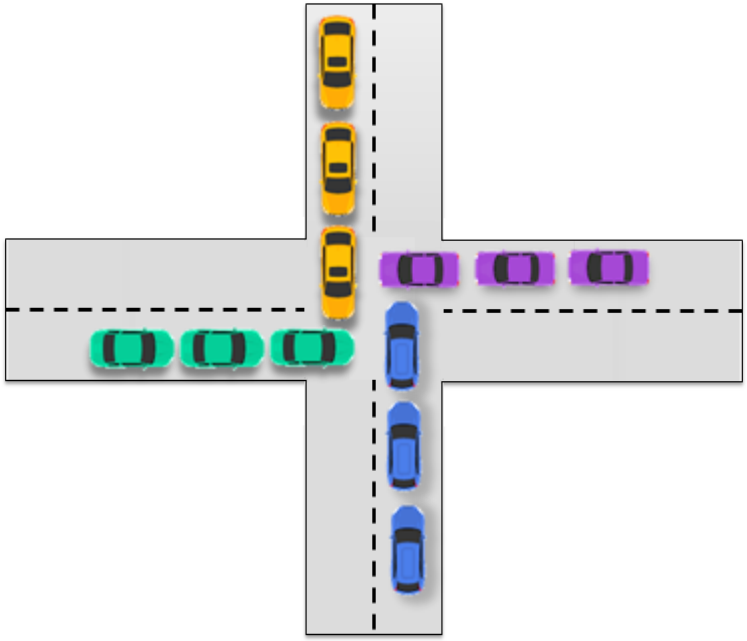
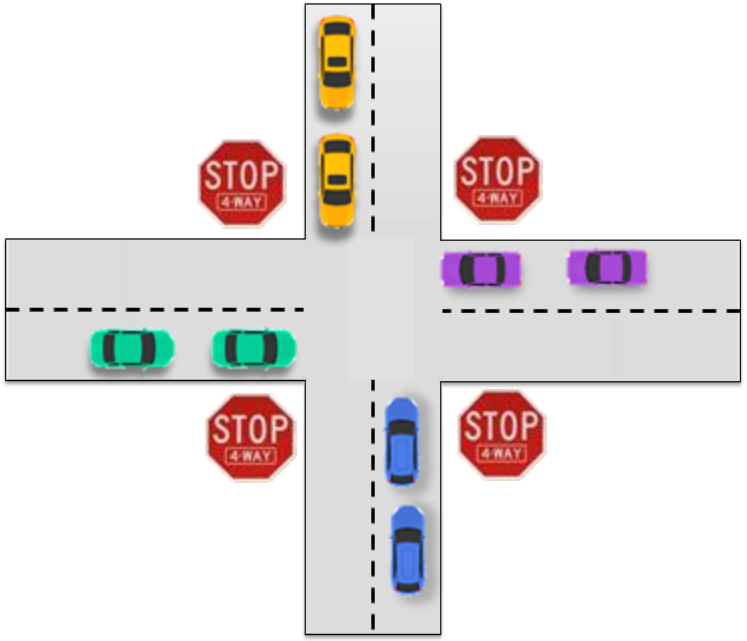
Deadlocks

Chapter 6

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Can occur in a variety of situations





Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Deadlock Definition

A set of processes is **deadlocked** if …

- Each process in the set waiting for an event
- That event can be caused only by another process

Deadlock occurs when all the members of a set of processes are blocked waiting for an event that only other members of the same set can cause. This situation causes all the processes to wait forever.

Deadlocks can occur on hardware resources or on software resources.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Example(hardware):

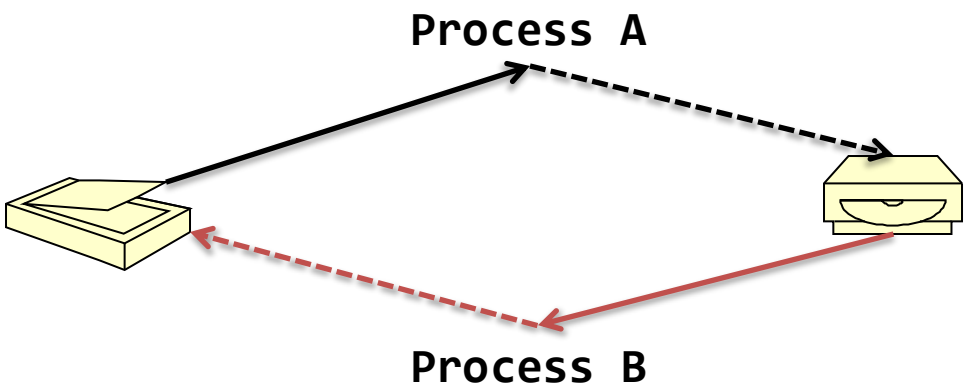
Process A requests permission to use the scanner and is granted it.

Process B is programmed differently and requests the Blu-ray recorder first and is also granted it.

Now A asks for the Blu-ray recorder, but the request is suspended until B releases it.

Unfortunately, instead of releasing the Blu-ray recorder, B asks for the scanner.

At this point both processes are blocked and will remain so forever. This situation is called a deadlock.

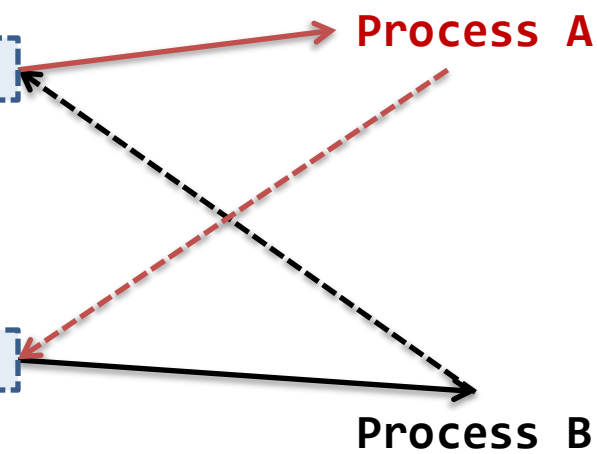


Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Example(software):

If process A locks record R1 and process B locks record R2, and then each process tries to lock the other one's record, we also have a deadlock.

65	207	1	1	10	35	11111111	101011111	1110
66	209	1	1	R1	11	10000100	100000111	0
67	215	0	1	9	24	11110111	100000111	0
68	223	1	1	17	29	11111111	0	1111
69	226	1	1	7	19	11110111	10011000	1
70	227	0	1	10	31	11111101	100110111	1001
71	228	0	1	24	9	10000100	100010000	100
72	231	0	1	2	25	11110111	100000111	1000
73	239	1	1	R2	23	11111111	100011111	101
74	245	0	1	18	13	11100101	10000	0



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

RESOURCES

- Preemptable and Nonpreemptable Resources
- Resource Acquisition

INTRODUCTION TO DEADLOCKS

- Conditions for Resource Deadlocks
- Deadlock Modeling

THE OSTRICH ALGORITHM

DEADLOCK DETECTION AND RECOVERY

- Deadlock Detection with One Resource of Each Type
- Deadlock Detection with Multiple Resources of Each Type
- Recovery from Deadlock
  - Recovery through Preemption
  - Recovery through Rollback
  - Recovery through Killing Processes

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Resource Trajectories
Safe and Unsafe States
The Banker's Algorithm for a Single Resource
The Banker's Algorithm for Multiple Resources

DEADLOCK PREVENTION

Attacking the Mutual-Exclusion Condition
Attacking the Hold-and-Wait Condition
Attacking the No-Preemption Condition
Attacking the Circular Wait Condition

OTHER ISSUES

Two-Phase Locking
Communication Deadlocks
Livelock
Starvation

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

A resource is anything that must be acquired, used, and released over the course of time

A resource can be a hardware device (e.g., a Blu-ray drive) or a piece of information (e.g., a record in a database).

For some resources, several identical instances may be available, such as three Blu-ray drives. When several copies of a resource are available, any one of them can be used to satisfy any request for the resource.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Preemptable and Nonpreemptable Resources

A preemptable resource is one that can be taken away from the process owning it with no ill effects.

A nonpreemptable resource cannot be taken away from its current owner without potentially causing failure.

In general, deadlocks involve nonpreemptable resources (will be considered here).

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Preemptable and Nonpreemptable Resources

Example(preemptable resource**):**

1 GB memory
2 1-GB Processes, each want to print something

Process A requests and gets the printer, then starts to compute the values to print. Before it has finished the computation, it exceeds its time quantum and is swapped out to disk.

Process B now runs and tries to acquire the printer.

Deadlock situation:
A has the printer and B has the memory, and neither one can proceed without the resource held by the other.

Solution:
Preempt (take away) the memory from B by swapping it out and swapping A in. Now A can run, do its printing, and then release the printer. No deadlock occurs.

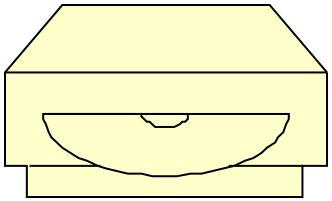
Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Preemptable and Nonpreemptable Resources

Example(nonpreemptable resource**):**

A process has begun to burn a Blu-ray, suddenly taking the Blu-ray recorder away from it and giving it to another process will result in a garbled Blu-ray.



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Preemptable and Nonpreemptable Resources

Sequence of events required to use a resource

1. Request the resource.
2. Use the resource.
3. Release the resource.

(we will assume that when a process is denied a resource request, it is put to sleep)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Resource Acquisition

One way of allowing this is to associate a semaphore with each resource.

These semaphores are all initialized to 1 (Mutexes can be used equally well).

The three steps listed above are then implemented as a down on the semaphore to acquire the resource, the use of the resource, and finally an up on the resource to release it.

```
down(s)
 if (s == 0){
    sleep
 }
 s--


up(s)
 s++
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Resource Acquisition

```
typedef int semaphore;
semaphore resource_1;

void process_A(void) {
        down(&resource_1);
        use_resource_1( );
        up(&resource_1);
}
```

(a)

```
typedef int semaphore;
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
}
```

(b)

(a) One resource. (b) Two resources.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Resource Acquisition

```
typedef int semaphore;
    semaphore resource_1;
    semaphore resource_2;

    void process_A(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
    }

    void process_B(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
    }
```

(a)

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_2);
    down(&resource_1);
    use_both_resources( );
    up(&resource_1);
    up(&resource_2);
}
```

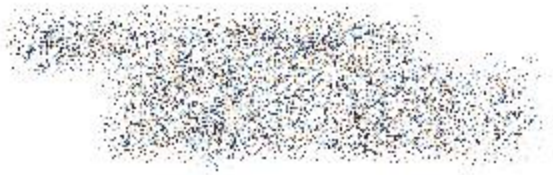
(b)

(a) Deadlock-free code. (b) Code with a potential deadlock.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Resources

Resource Acquisition



Deadlock

Process A acquires resource 1
Process B acquires resource 2

Each one will now block when trying to acquire the other one. Neither process will ever run again.

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
        down(&resource_1);
        down(&resource_2);
        use_both_resources( );
        up(&resource_2);
        up(&resource_1);
}

void process_B(void) {
        down(&resource_2);
        down(&resource_1);
        use_both_resources( );
        up(&resource_1);
        up(&resource_2);
}
```

(b)

(a) Deadlock-free code. (b) Code with a potential deadlock.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

RESOURCES

- Preemptable and Nonpreemptable Resources
- Resource Acquisition

INTRODUCTION TO DEADLOCKS

- Conditions for Resource Deadlocks
- Deadlock Modeling

THE OSTRICH ALGORITHM

DEADLOCK DETECTION AND RECOVERY

- Deadlock Detection with One Resource of Each Type
- Deadlock Detection with Multiple Resources of Each Type
- Recovery from Deadlock
  - Recovery through Preemption
  - Recovery through Rollback
  - Recovery through Killing Processes

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

INTRODUCTION TO DEADLOCKS

Conditions for Resource Deadlocks

All the processes are waiting, none of them will ever cause any event that could wake up any of the other members of the set, and all the processes continue to wait forever.

For this model, we assume that processes are single threaded and that no interrupts are possible to wake up a blocked process.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

INTRODUCTION TO DEADLOCKS

Conditions for Resource Deadlocks

Four conditions must hold for there to be a resource deadlock:

1. Mutual exclusion

   Each resource is either currently assigned to exactly one process or is available

2. Hold and wait

   Processes currently holding resources that were granted earlier can request new resources.

3. No preemption

   Resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them.

4. Circular wait condition

   There must be a circular list of two or more processes, each of which is waiting for a resource held by the next member of the chain.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

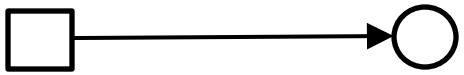
INTRODUCTION TO DEADLOCKS

Deadlock Modeling

Four conditions can be modeled using directed graphs. The graphs have two kinds of nodes:

1) processes, shown as circles
2) resources, shown as squares

A directed arc from a resource node to a process node means that the resource has previously been requested by, granted to, and is currently held by that process.

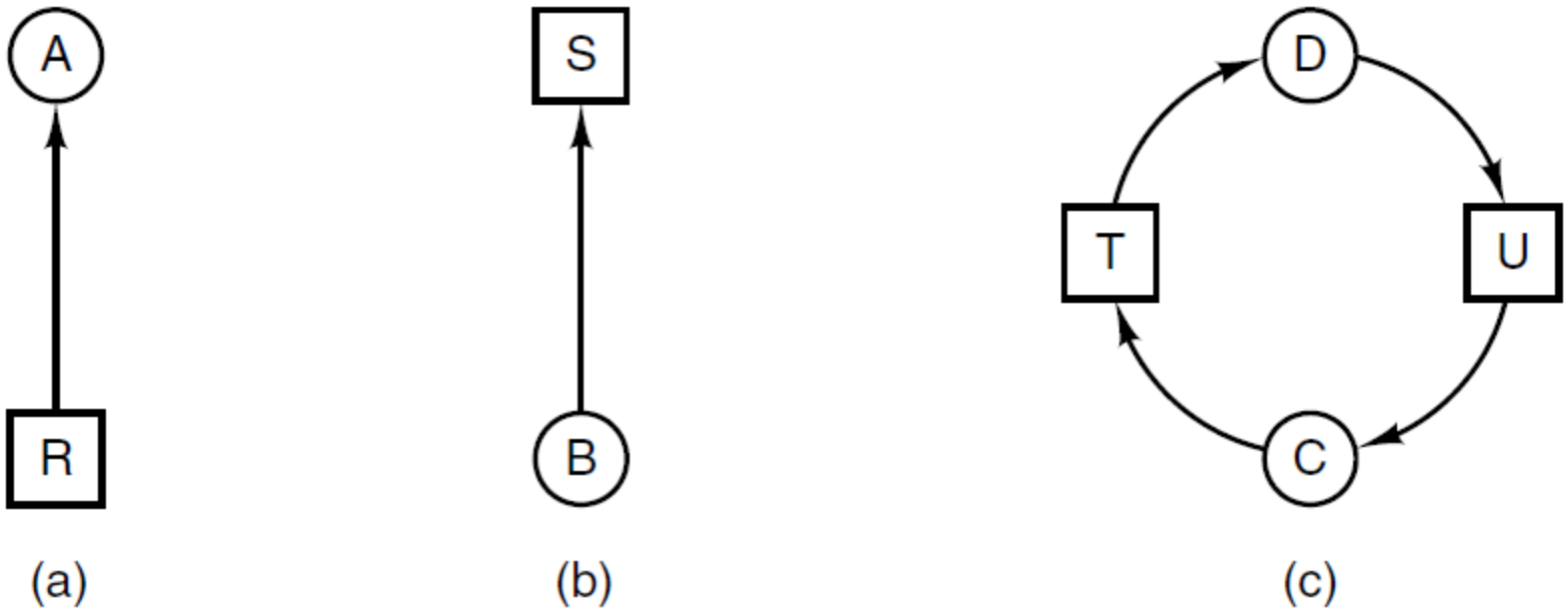


A directed arc from a process to a resource means that the process is currently blocked waiting for that resource.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

INTRODUCTION TO DEADLOCKS

Deadlock Modeling



Resource allocation graphs.
(a) Holding a resource. (b) Requesting a resource. (c) Deadlock.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

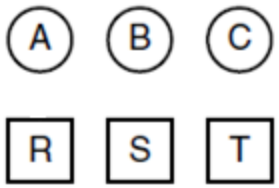
INTRODUCTION TO DEADLOCKS

Deadlock Modeling

The operating system is free to run any unblocked process at any instant, e.g. runs A until A finished all its work, then runs B to completion, and finally runs C.

This ordering does not lead to any deadlocks (because there is no competition for resources) but it also has no parallelism at all.

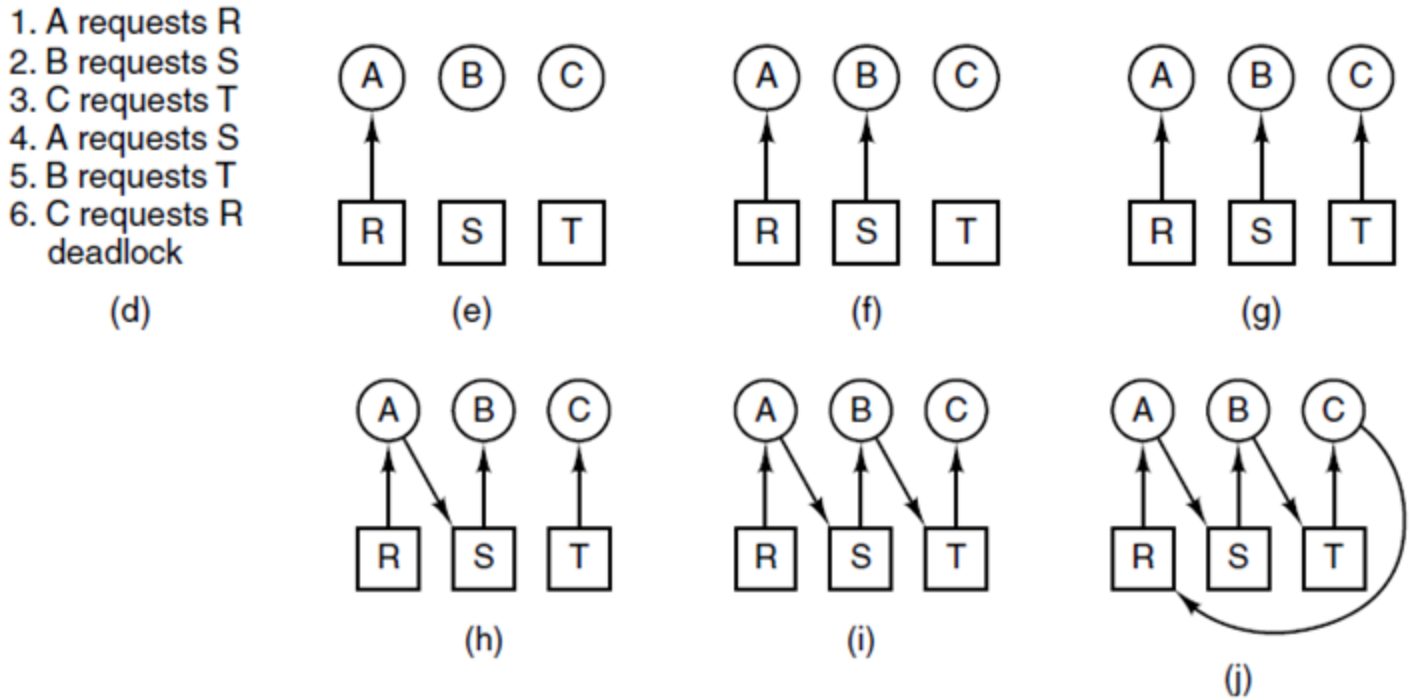
A	B	C
Request R	Request S	Request T
Request S	Request T	Request R
Release R	Release S	Release T
Release S	Release T	Release R
(a)	(b)	(c)



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

INTRODUCTION TO DEADLOCKS

Deadlock Modeling

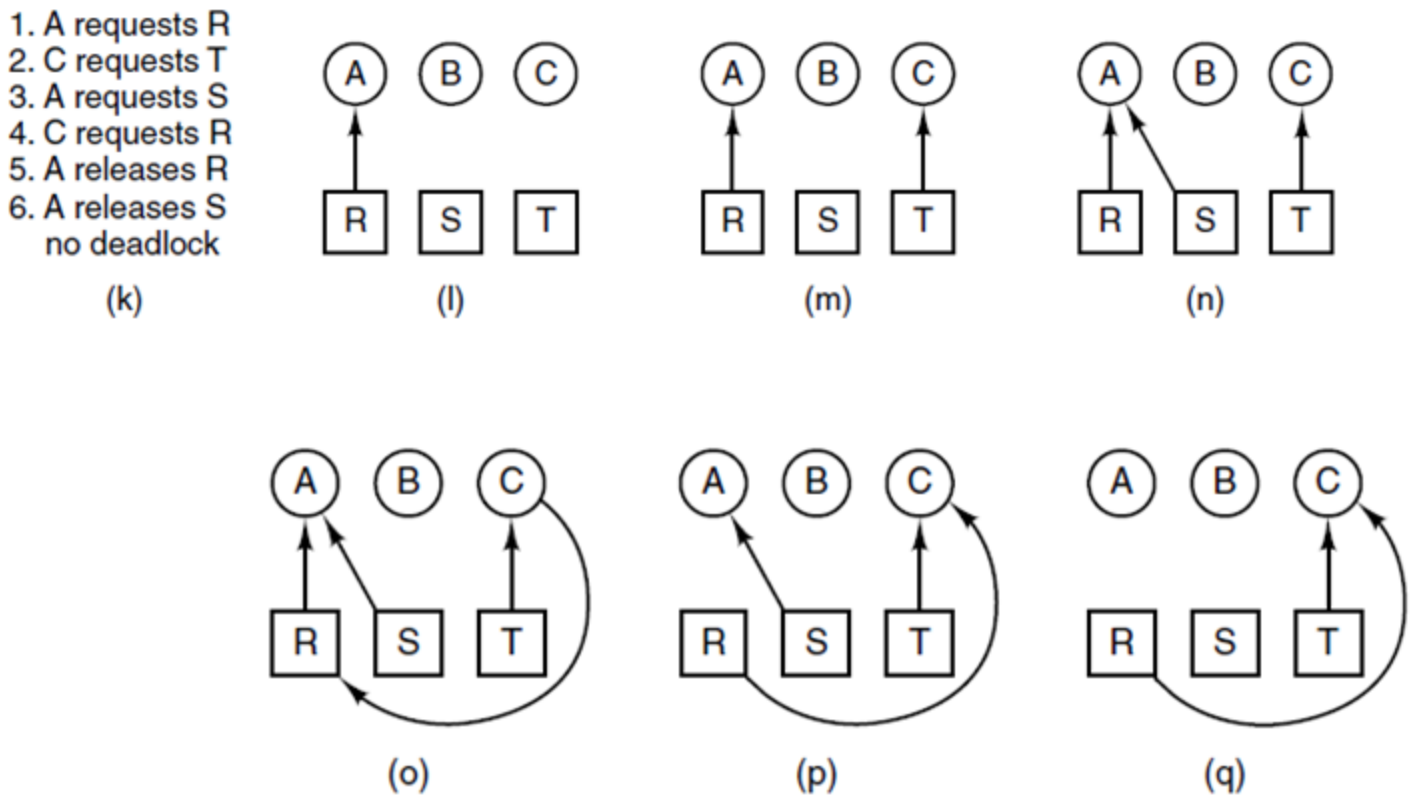


An example of how **deadlock** occurs and how it can be avoided.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

INTRODUCTION TO DEADLOCKS

Deadlock Modeling



An example of how **deadlock** occurs and how it can be **avoided**.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

INTRODUCTION TO DEADLOCKS

Deadlock Modeling

Strategies are used for dealing with deadlocks:

1. Ignore the problem, maybe it will go away.
2. Detection and recovery. Let deadlocks occur, detect them, and take action.
3. Dynamic avoidance by careful resource allocation.
4. Prevention, by structurally negating one of the four required conditions.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

RESOURCES

- Preemptable and Nonpreemptable Resources
- Resource Acquisition

INTRODUCTION TO DEADLOCKS

- Conditions for Resource Deadlocks
- Deadlock Modeling

THE OSTRICH ALGORITHM

DEADLOCK DETECTION AND RECOVERY

- Deadlock Detection with One Resource of Each Type
- Deadlock Detection with Multiple Resources of Each Type
- Recovery from Deadlock
  - Recovery through Preemption
  - Recovery through Rollback
  - Recovery through Killing Processes

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

THE OSTRICH ALGORITHM

Ostrich algorithm:

Stick your head in the sand and pretend there is no problem.

Mathematicians find it unacceptable and say that deadlocks must be prevented at all costs.

Engineers ask how often the problem is expected. If deadlocks occur on the average once every five years, just ignore it.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

RESOURCES

- Preemptable and Nonpreemptable Resources
- Resource Acquisition

INTRODUCTION TO DEADLOCKS

- Conditions for Resource Deadlocks
- Deadlock Modeling

THE OSTRICH ALGORITHM

DEADLOCK DETECTION AND RECOVERY

- Deadlock Detection with One Resource of Each Type
- Deadlock Detection with Multiple Resources of Each Type
- Recovery from Deadlock
  - Recovery through Preemption
  - Recovery through Rollback
  - Recovery through Killing Processes

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

There is only one resource of each type e.g. one printer, one scanner, one Blu-ray recorder, one plotter, and one tape drive, but no more than one of each class of resource.

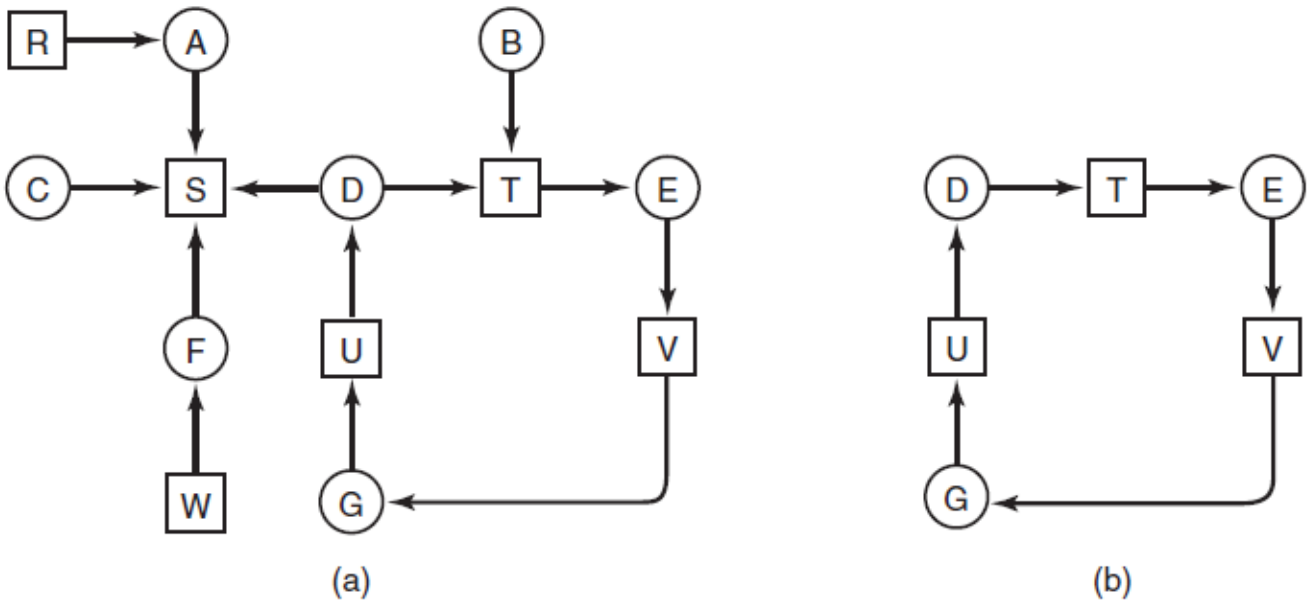
Example of a system – is it deadlocked?

1. Process A holds R, wants S
2. Process B holds nothing, wants T
3. Process C holds nothing, wants S
4. Process D holds U, wants S and T
5. Process E holds T, wants V
6. Process F holds W, wants S
7. Process G holds V, wants U

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type



(a) A resource graph. (b) A cycle extracted from (a).

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

For each node, *N* in the graph, perform following five steps with *N* as starting node.

1. Initialize *L* to empty list, and designate all arcs as unmarked.
2. Add current node to end of L, check to see if node now appears in L two times. If so, graph contains a cycle (listed in L) and algorithm terminates
3. From given node, see if there are any unmarked outgoing arcs. If so, go to step 4; if not, go to step 5.
4. Pick unmarked outgoing arc at random, mark it. Then follow to new current node and go to step 2.
5. If this is initial node, graph does not contain cycles, algorithm terminates. Otherwise, dead end. Remove it and go back to the previous node.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

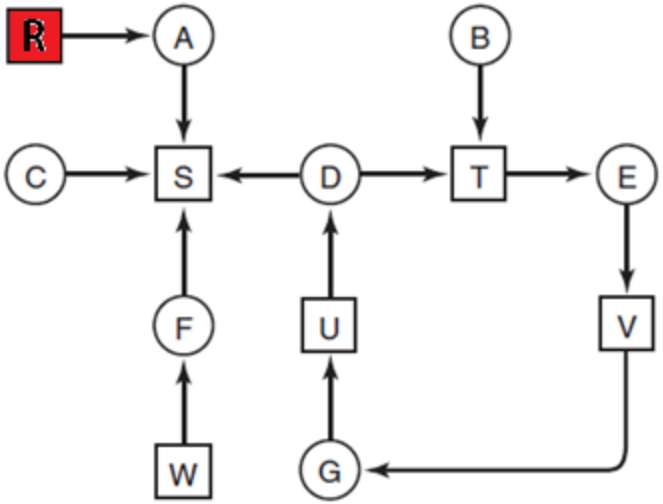
Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

L=[]

-> R : L=[R]



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

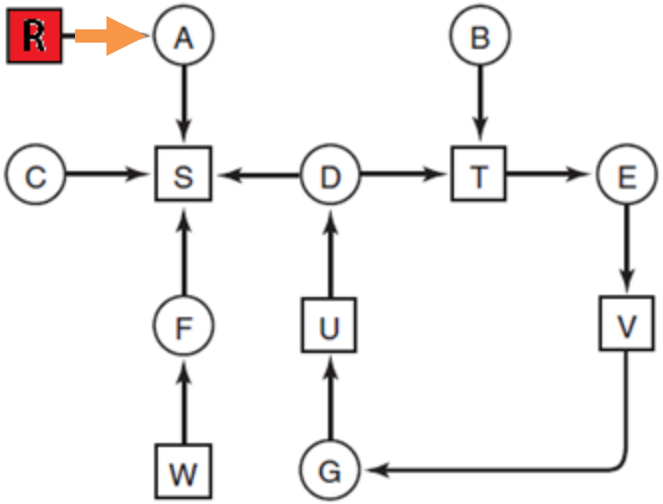
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

```
L=[]
-> R : L=[R]
-> A : L=[R, A]
```



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

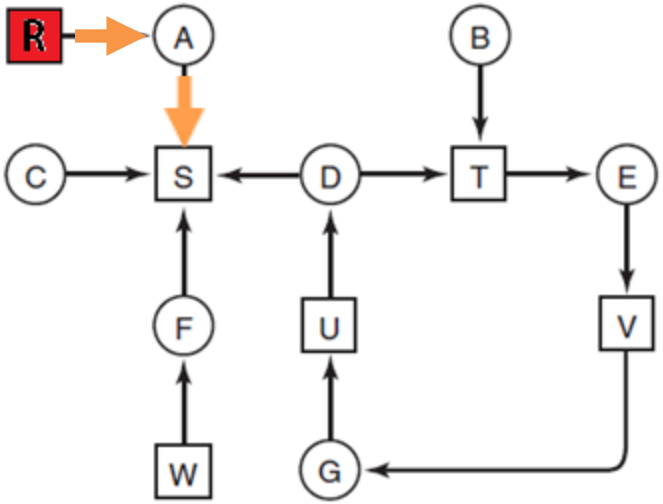
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

L=[]
-> R : L=[R]
-> A : L=[R, A]
-> S : L=[R, A, S]
L=[R, A]



S has no outgoing arcs, so it is a dead end.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

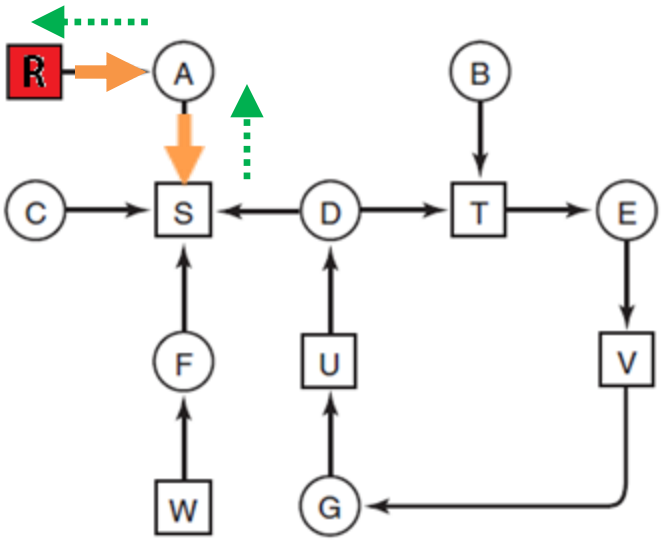
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

```
L=[]
-> R : L=[R]
-> A : L=[R, A]
-> S : L=[R, A, S]
A <- : L=[R, A]
R <- : L=[R, A]
```



Forcing us to backtrack to A. Since A has no unmarked outgoing arcs, we backtrack to R, completing our inspection of R.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

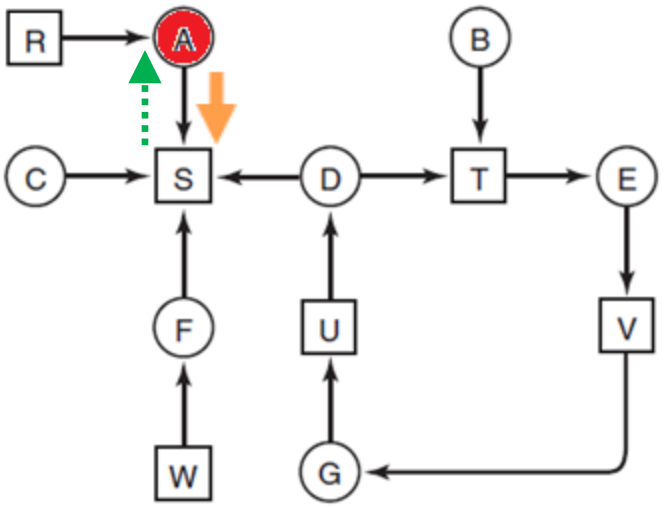
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

L=[]
-> A : L=[A]
-> S : L=[A, S]
L=[A]
A <- : L=[A]



S has no outgoing arcs, so it is a dead end. we backtrack to A, completing our inspection of A.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

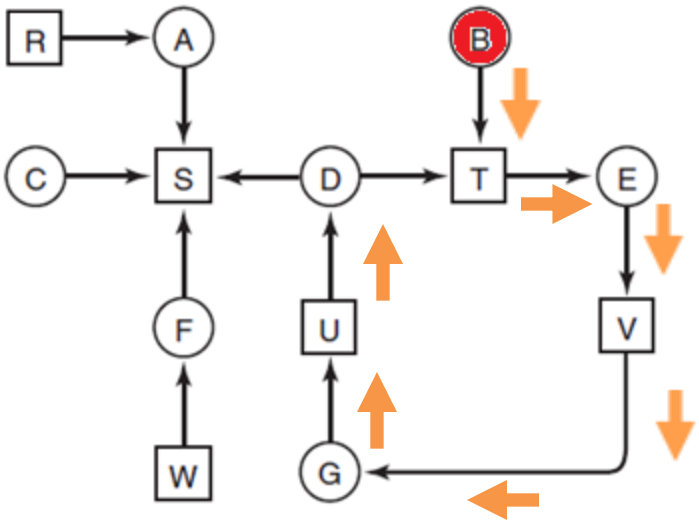
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

```
L=[]
-> B : L=[B]
-> T : L=[B, T]
-> E : L=[B, T, E]
-> V : L=[B, T, E, V]
-> G : L=[B, T, E, V, G]
-> U : L=[B, T, E, V, G, U]
-> D : L=[B, T, E, V, G, U, D]
```



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

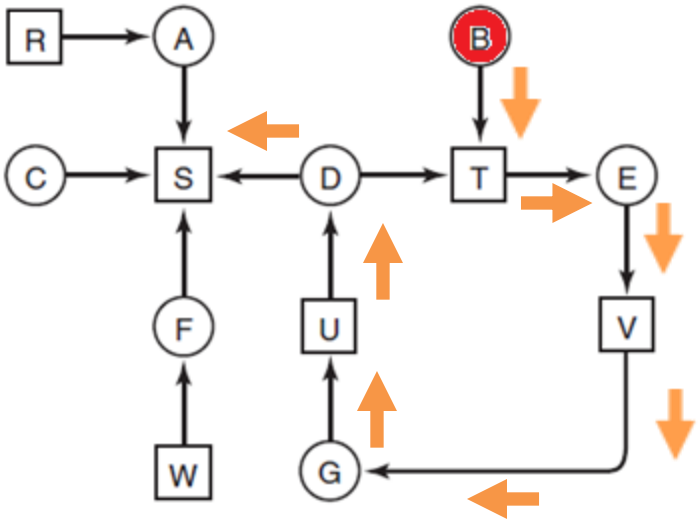
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

```
L=[]
-> B : L=[B]
-> T : L=[B, T]
-> E : L=[B, T, E]
-> V : L=[B, T, E, V]
-> G : L=[B, T, E, V, G]
-> U : L=[B, T, E, V, G, U]
-> D : L=[B, T, E, V, G, U, D]
-> S : L=[B, T, E, V, G, U, D, S]
```



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

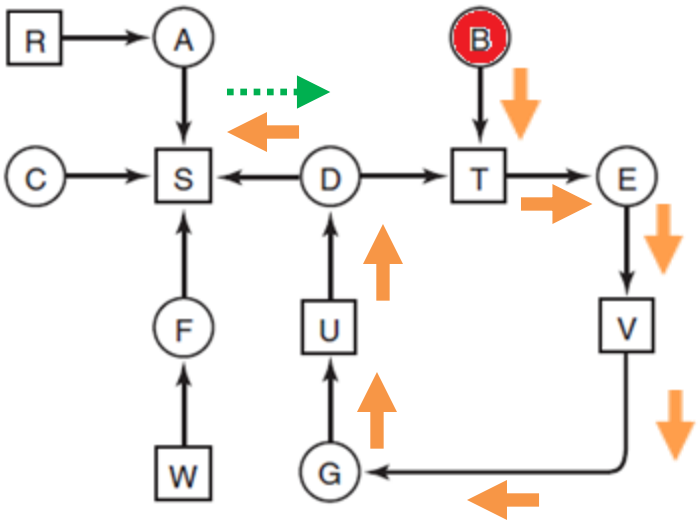
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

```
L=[]
-> B : L=[B]
-> T : L=[B, T]
-> E : L=[B, T, E]
-> V : L=[B, T, E, V]
-> G : L=[B, T, E, V, G]
-> U : L=[B, T, E, V, G, U]
-> D : L=[B, T, E, V, G, U, D]
-> S : L=[B, T, E, V, G, U, D, S]
L=[B, T, E, V, G, U, D]
-> D : =[B, T, E, V, G, U, D]
```



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

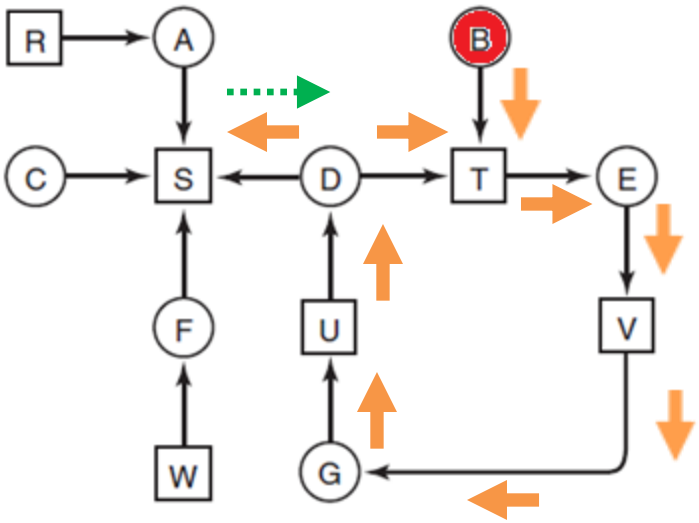
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

L=[ ]
-> B : L=[B]
-> T : L=[B, T]
-> E : L=[B, T, E]
-> V : L=[B, T, E, V]
-> G : L=[B, T, E, V, G]
-> U : L=[B, T, E, V, G, U]
-> D : L=[B, T, E, V, G, U, D]
-> S : L=[B, T, E, V, G, U, D, S]
L=[B, T, E, V, G, U, D]
-> D : L=[B, T, E, V, G, U, D]
-> T : L=[B, T, E, V, G, U, D, T]



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

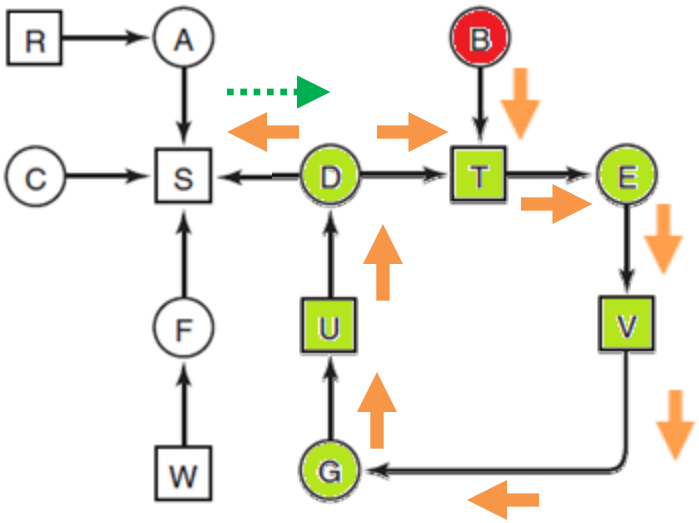
DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with One Resource of Each Type

Algorithm to Detect Deadlocks

Example :

L=[ ]
-> B : L=[B]
-> T : L=[B, T]
-> E : L=[B, T, E]
-> V : L=[B, T, E, V]
-> G : L=[B, T, E, V, G]
-> U : L=[B, T, E, V, G, U]
-> D : L=[B, T, E, V, G, U, D]
-> S : L=[B, T, E, V, G, U, D, S]
L=[B, T, E, V, G, U, D]
-> D : L=[B, T, E, V, G, U, D]
-> T : L=[B, T, E, V, G, U, D, T]



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

When multiple copies of some of the resources exist, a different approach is needed to detect deadlocks. A matrix-based algorithm is used to detect deadlock among n processes, P1 through Pn.

A: the available resource vector (A1, A2, ...)

with Ai giving the number of instances of resource i that are currently available (unassigned). E.g. If both of our two tape drives are assigned, A1 will be 0.

E: the existing resource vector. (E1, E2, ...)

E1 resources of class 1, E2 resources of class 2, ...

e.g. if class 1 is tape drives, then E1 = 2 means the system has two tape drives.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

C : the current allocation matrix

Cij is the number of instances of resource j that are held by process i.

R : the request matrix

Rij is the number of instances of resource j that Pi wants.

In this model when a process asks for resources, it asks for them all at once (the request matrix, R).

Every resource is either allocated or is available:

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

Resources in existence
$(E_1, E_2, E_3, \ldots, E_m)$

Resources available
$(A_1, A_2, A_3, \ldots, A_m)$

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \cdots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \cdots & C_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \cdots & C_{nm} \end{bmatrix}$$

Row n is current allocation to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \cdots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \cdots & R_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \cdots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2 needs

The four data structures needed by the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

Deadlock detection algorithm:

1. Look for unmarked process, P_i , for which the i-th row of R is less than or equal to A.
2. If such a process is found, add the i-th row of C to A, mark the process, go back to step 1.
3. If no such process exists, algorithm terminates.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

Example :

Three processes and four resource classes (tape drives, plotters, scanners, and Blu-ray drives)

Process 1 :

one scanner

Process 2 :

two tape drives and a Blu-ray drive.

Process 3 :

a plotter and two scanners.

Each process needs additional resources, as shown by the R matrix.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

E =

Tape drives	Plotters	Scanners	Blu-rays
4	2	3	1

A =

Tape drives	Plotters	Scanners	Blu-rays
2	1	0	0

C =

	Tape drives	Plotters	Scanners	Blu-rays
P1	0	0	1	0
P2	2	0	0	1
P3	0	1	2	0

R =

	Tape drives	Plotters	Scanners	Blu-rays
P1	2	0	0	1
P2	1	0	1	0
P3	2	1	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

$$\sum_{i=1}^{n} C_{ij} + A_j = E_j$$

	Tape drives	Plotters	Scanners	Blu-rays
E =	4	2	3	1

	Tape drives	Plotters	Scanners	Blu-rays
A =	2	1	0	0

C =	Tape drives	Plotters	Scanners	Blu-rays
P1	0	0	1	0
P2	2	0	0	1
P3	0	1	2	0

R =	Tape drives	Plotters	Scanners	Blu-rays
P1	2	0	0	1
P2	1	0	1	0
P3	2	1	0	0

```
Resource 1 (Tape drive): C11 + C21 + C31 + A1 = E1
                          0   +  2  +  0  + 2 = 4
Resource 3 (Scanners)  : C13 + C23 + C33 + A3 = E3
                          1   +  0  +  2  + 0 = 3
Resource 2 (?)         :  ? + ? + ? + ? = ?
                         ? + ? + ? + ? = ?
Resource 4 (?)         :  ? + ? + ? + ? = ?
                         ? + ? + ? + ? = ?
```

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

E =

Tape drives	Plotters	Scanners	Blu-rays
4	2	3	1

A =

Tape drives	Plotters	Scanners	Blu-rays
2	1	0	0

C =

	Tape drives	Plotters	Scanners	Blu-rays
P1	0	0	1	0
P2	2	0	0	1
P3	0	1	2	0

R =

	Tape drives	Plotters	Scanners	Blu-rays
✕P1	2	0	0	1
P2	1	0	1	0
P3	2	1	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

$$E = \begin{array}{|c|c|c|c|} \hline \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline 4 & 2 & 3 & 1 \\ \hline \end{array}$$

$$A = \begin{array}{|c|c|c|c|} \hline \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline 2 & 1 & 0 & 0 \\ \hline \end{array}$$

$$C = \begin{array}{c|c|c|c|c|} & \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline P1 & 0 & 0 & 1 & 0 \\ \hline P2 & 2 & 0 & 0 & 1 \\ \hline P3 & 0 & 1 & 2 & 0 \\ \hline \end{array}$$

$$R = \begin{array}{c|c|c|c|c|} & \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline \times P1 & 2 & 0 & 0 & 1 \\ \hline \times P2 & 1 & 0 & 1 & 0 \\ \hline P3 & 2 & 1 & 0 & 0 \\ \hline \end{array}$$

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

$$E = \begin{array}{|c|c|c|c|} \hline \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline 4 & 2 & 3 & 1 \\ \hline \end{array}$$

$$A = \begin{array}{|c|c|c|c|} \hline \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline 2 & 1 & 0 & 0 \\ \hline \end{array}$$

$$C = \begin{array}{c|c|c|c|c|} & \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline \text{P1} & 0 & 0 & 1 & 0 \\ \text{P2} & 2 & 0 & 0 & 1 \\ \text{P3} & 0 & 1 & 2 & 0 \\ \hline \end{array}$$

$$R = \begin{array}{c|c|c|c|c|} & \text{Tape drives} & \text{Plotters} & \text{Scanners} & \text{Blu-rays} \\ \hline \text{P1} & 2 & 0 & 0 & 1 \\ \text{P2} & 1 & 0 & 1 & 0 \\ \text{P3} & 2 & 1 & 0 & 0 \\ \hline \end{array}$$

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

	Tape drives	Plotters	Scanners	Blu-rays
E =	4	2	3	1

	Tape drives	Plotters	Scanners	Blu-rays
A =	2	1	0	0

$$A_{new} = C_{p_3} + A_{old} = [0\ 1\ 2\ 0] + [2\ 1\ 0\ 0] = [2\ 2\ 2\ 0]$$

C =				
P1	0	0	1	0
P2	2	0	0	1
~~P3~~	~~0~~	~~1~~	~~2~~	~~0~~

R =				
P1	2	0	0	1
P2	1	0	1	0
P3	2	1	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

E =

Tape drives	Plotters	Scanners	Blu-rays
4	2	3	1

A =

Tape drives	Plotters	Scanners	Blu-rays
2	2	2	0

C =

	Tape drives	Plotters	Scanners	Blu-rays
P1	0	0	1	0
P2	2	0	0	1
P3	0	0	0	0

R =

	Tape drives	Plotters	Scanners	Blu-rays
P1	2	0	0	1
P2	1	0	1	0
P3	0	0	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

E =

Tape drives	Plotters	Scanners	Blu-rays
4	2	3	1

A =

Tape drives	Plotters	Scanners	Blu-rays
2	2	2	0

C =

	Tape drives	Plotters	Scanners	Blu-rays
P1	0	0	1	0
P2	2	0	0	1
P3	0	0	0	0

R =

	Tape drives	Plotters	Scanners	Blu-rays
P1	2	0	0	1
P2	1	0	1	0
P3	2	1	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

$E =$

Tape drives	Plotters	Scanners	Blu-rays
4	2	3	1

$A =$

Tape drives	Plotters	Scanners	Blu-rays
2	2	2	0

$C =$

	Tape drives	Plotters	Scanners	Blu-rays
P1	0	0	1	0
P2	2	0	0	1
P3	0	0	0	0

R

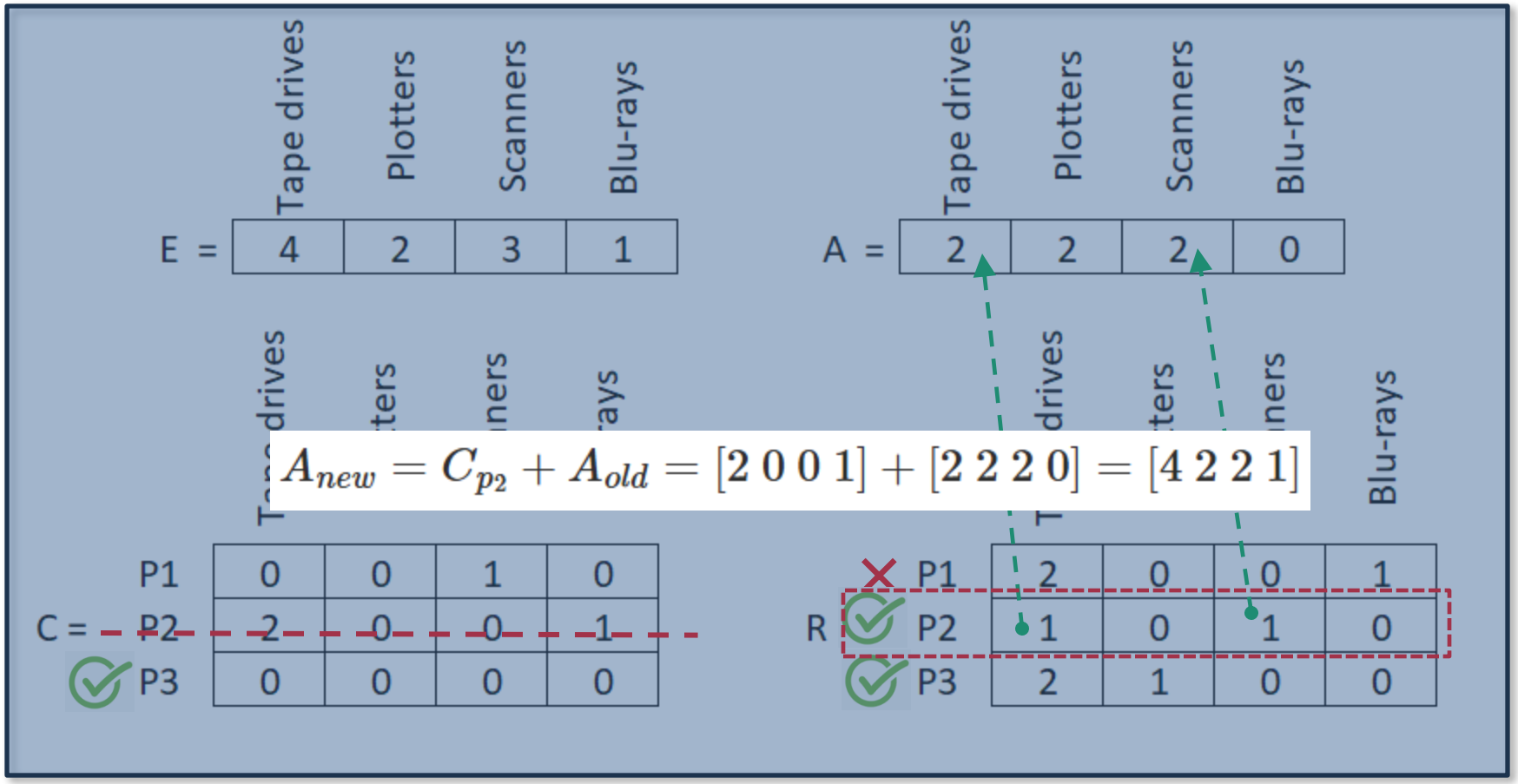
	Tape drives	Plotters	Scanners	Blu-rays
P1	2	0	0	1
P2	1	0	1	0
P3	2	1	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type



An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

	Tape drives	Plotters	Scanners	Blu-rays
E =	4	2	3	1

	Tape drives	Plotters	Scanners	Blu-rays
A =	4	2	2	1

		Tape drives	Plotters	Scanners	Blu-rays
	P1	0	0	1	0
C	P2	0	0	0	0
	P3	0	0	0	0

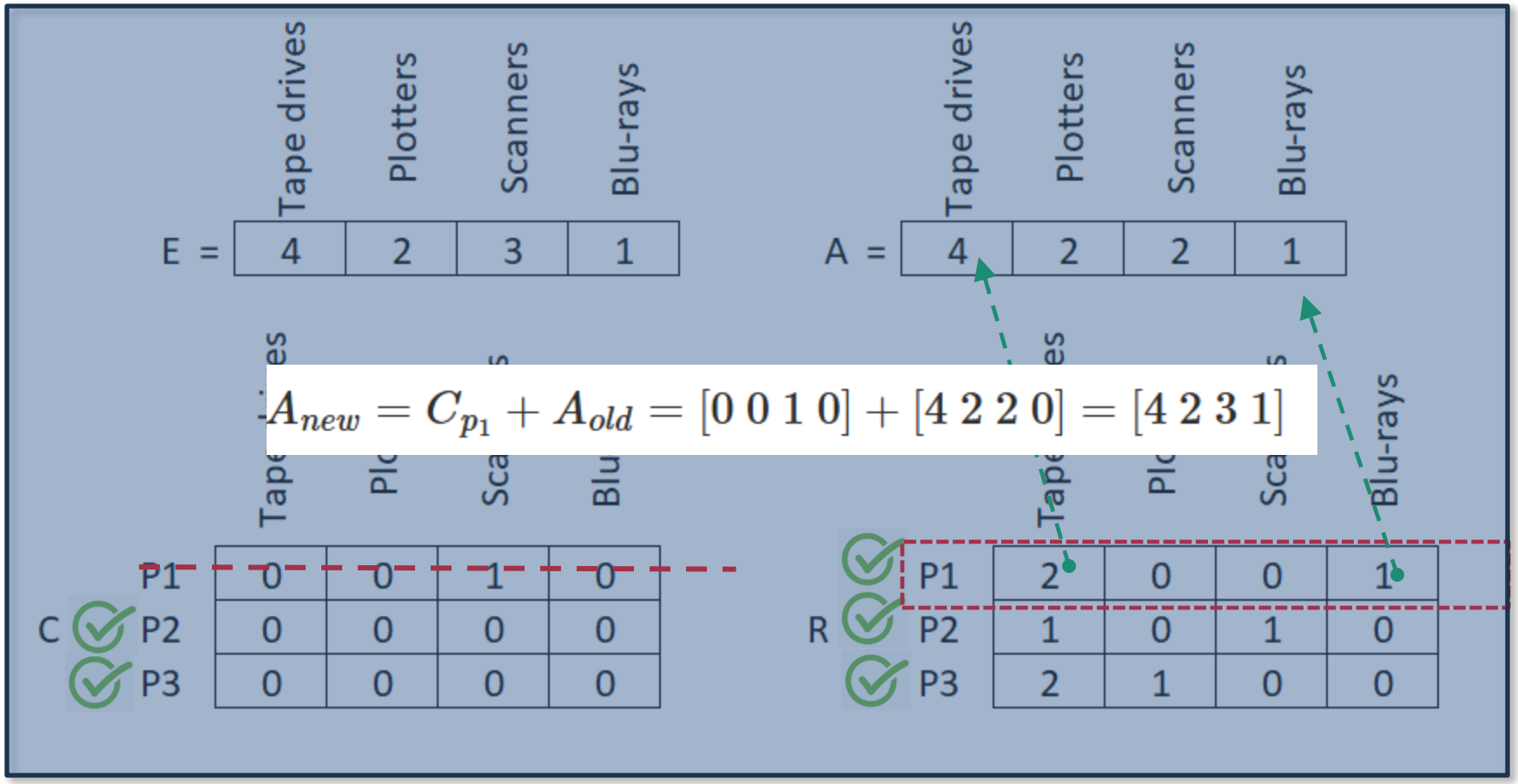
		Tape drives	Plotters	Scanners	Blu-rays
	P1	2	0	0	1
R	P2	1	0	1	0
	P3	2	1	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type



An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

E =

Tape drives	Plotters	Scanners	Blu-rays
4	2	3	1

A =

Tape drives	Plotters	Scanners	Blu-rays
4	2	3	1

No deadlock

C

	Tap	Pl	Sca	Blu
P1	0	0	0	0
P2	0	0	0	0
P3	0	0	0	0

R

	Tap	Pl	Sca	Blu
P1	2	0	0	1
P2	1	0	1	0
P3	2	1	0	0

An example for the deadlock detection algorithm.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Deadlock Detection with Multiple Resources of Each Type

	Tape drives	Plotters	Scanners	Blu-rays
E =	4	2	3	1

	Tape drives	Plotters	Scanners	Blu-rays
A =	2	1	0	0

Deadlock ?

		Tape drives	Plotters	Scanners	Blu-rays
C =	P1	0	0	1	0
	P2	2	0	0	1
	P3	0	1	2	0

		Tape drives	Plotters	Scanners	Blu-rays
R =	P1	2	0	0	1
	P2	1	0	1	0
	P3	2	1	0	1

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Recovery from Deadlock

Suppose that our deadlock detection algorithm has succeeded and detected a deadlock. What next? Some way is needed to recover and get the system going again.

Possible Methods of recovery:

1. Preemption
2. Rollback (simply means abort the process and restart it)
3. Killing processes

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Recovery from Deadlock

Preemption

- Take a resource from some other process.

  In some cases it may be possible to temporarily take a resource away from its current owner and give it to another process.

- Depends on nature of the resource.

Recovery through Rollback

- Checkpoint a process periodically

  Checkpointing a process means that its state (e.g. the memory image and the resource state) is written to a file so that it can be restarted later. New checkpoints should not overwrite old ones but should be written to new files.

- Recovery

  To do the recovery, a process that owns a needed resource is rolled back to a point in time before it acquired that resource by starting at one of its earlier checkpoints.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Recovery from Deadlock

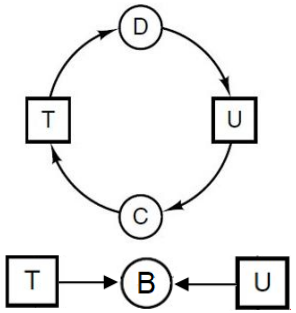
Killing processes

The crudest but simplest way to break a deadlock is to kill one or more processes (e.g., a process in the cycle).

With a little luck, the other processes will be able to continue. If this does not help, it can be repeated until the cycle is broken.

Alternatively, a process not in the cycle can be chosen as the victim in order to release its resources. The process to be killed is carefully chosen because it is holding resources that some process in the cycle needs.

A third process (B) may hold another identical **T** and another identical **U** and be happily running. Killing the third process will release these resources and break the deadlock involving the first two (D and C).



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK DETECTION AND RECOVERY

Recovery from Deadlock

Killing processes

It is best to kill a process that can be rerun from the beginning with no ill effects.

For example, a compilation can always be rerun because all it does is read a source file and produce an object file. If it is killed partway through, the first run has no influence on the second run.

On the other hand, a process that updates a database cannot always be run a second time safely.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Resource Trajectories
Safe and Unsafe States
The Banker's Algorithm for a Single Resource
The Banker's Algorithm for Multiple Resources

DEADLOCK PREVENTION

Attacking the Mutual-Exclusion Condition
Attacking the Hold-and-Wait Condition
Attacking the No-Preemption Condition
Attacking the Circular Wait Condition

OTHER ISSUES

Two-Phase Locking
Communication Deadlocks
Livelock
Starvation

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

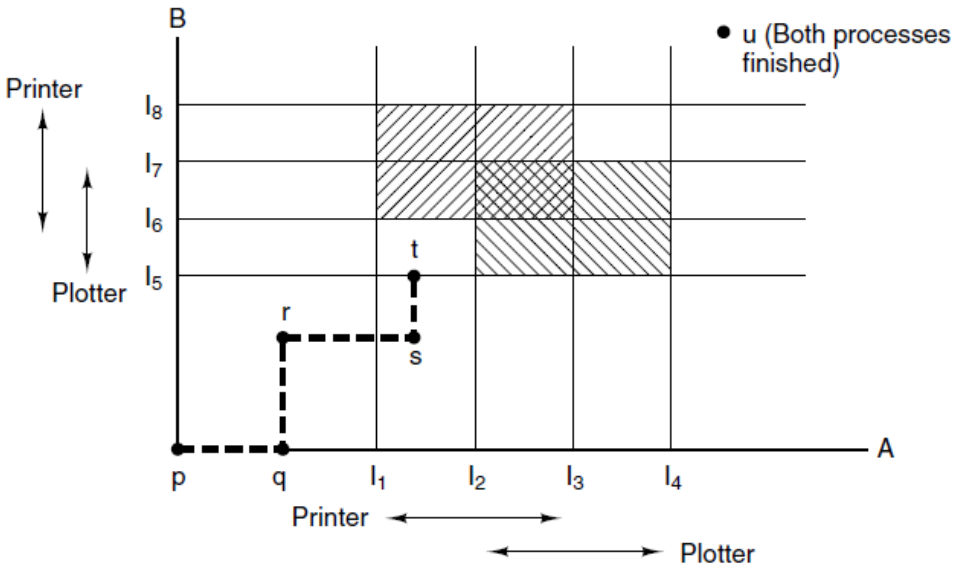
Resource Trajectories

<u>Process A</u> requests a printer at I1; at I2 it needs a plotter. The printer and plotter are released at I3 and I4.

The horizontal axis :
the number of instructions executed by **process A.**

<u>Process B</u> needs the plotter from I5 to I7 and the printer from I6 to I8.

The vertical axis :
the number of instructions executed by **process B.**



Two process resource trajectories.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

===

DEADLOCK AVOIDANCE

Resource Trajectories(Single processor)

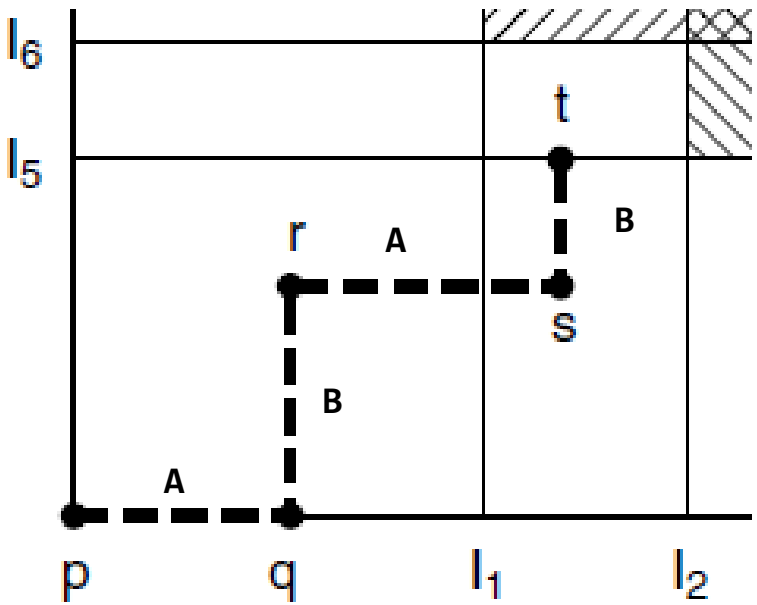
State at p: neither process having executed any instructions.

p → q : process A is selected by scheduler and has executed some number of instructions.

q → r : process B is selected by scheduler and has executed some number of instructions.

r → s : process A requests and is granted the printer.

s → t : When process B reaches point t, it requests the plotter.



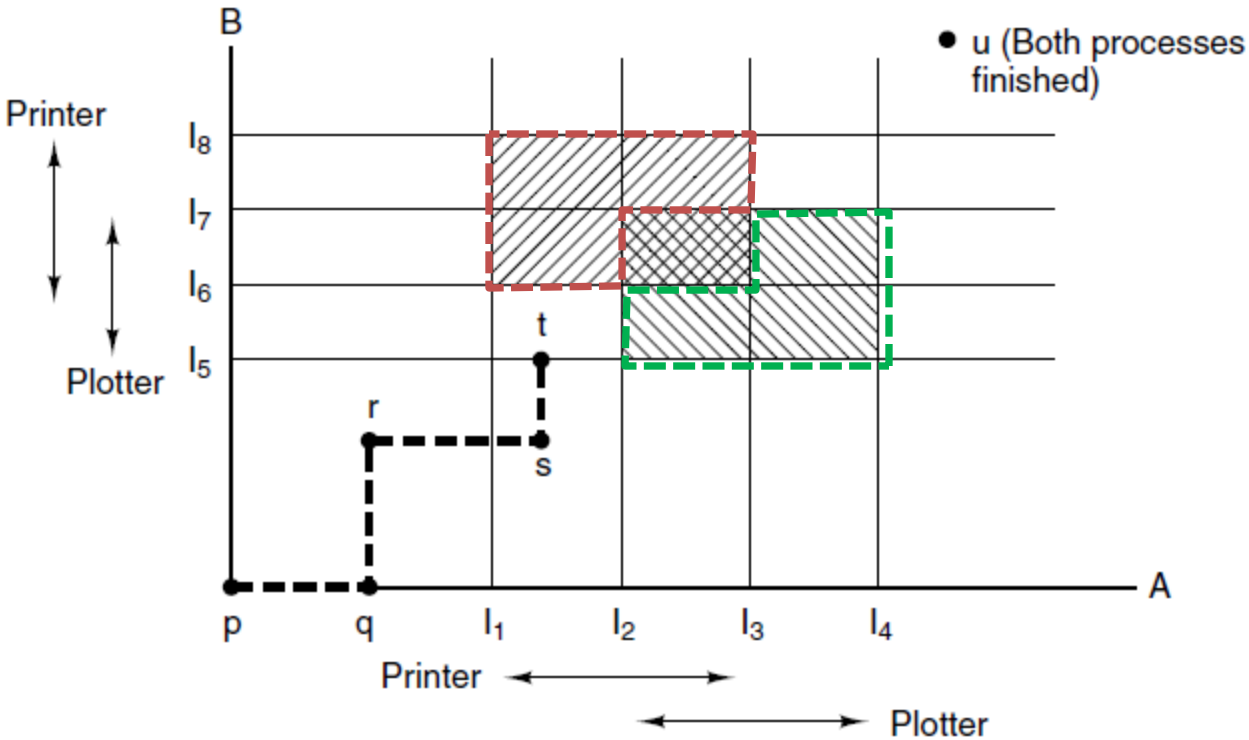
Two process resource trajectories.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

===

DEADLOCK AVOIDANCE

Resource Trajectories



Both processes have the printer
The mutual exclusion rule makes it impossible to enter this region

Both processes have the plotter
The mutual exclusion rule makes it impossible to enter this region

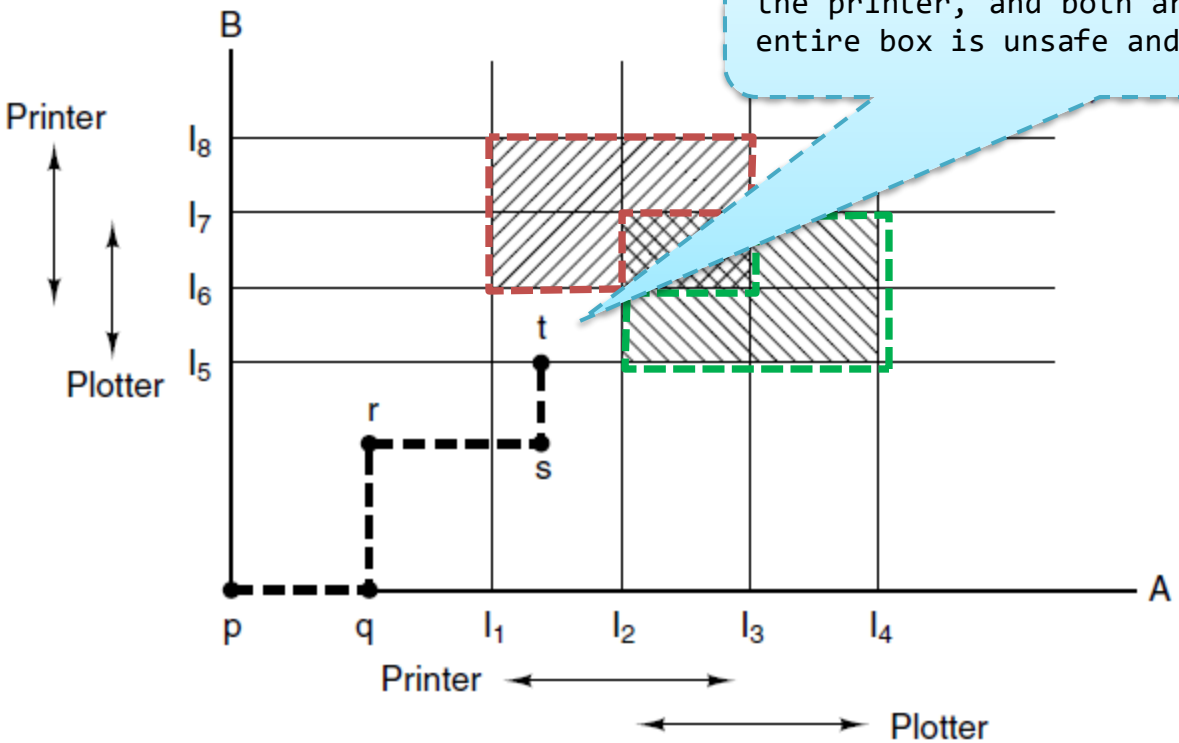
Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

===

DEADLOCK AVOIDANCE

Resource Trajectories

A is requesting the plotter and B is requesting the printer, and both are already assigned. The entire box is unsafe and must not be entered



Both processes have the printer
The mutual exclusion rule makes it impossible to enter this region

Both processes have the plotter
The mutual exclusion rule makes it impossible to enter this region

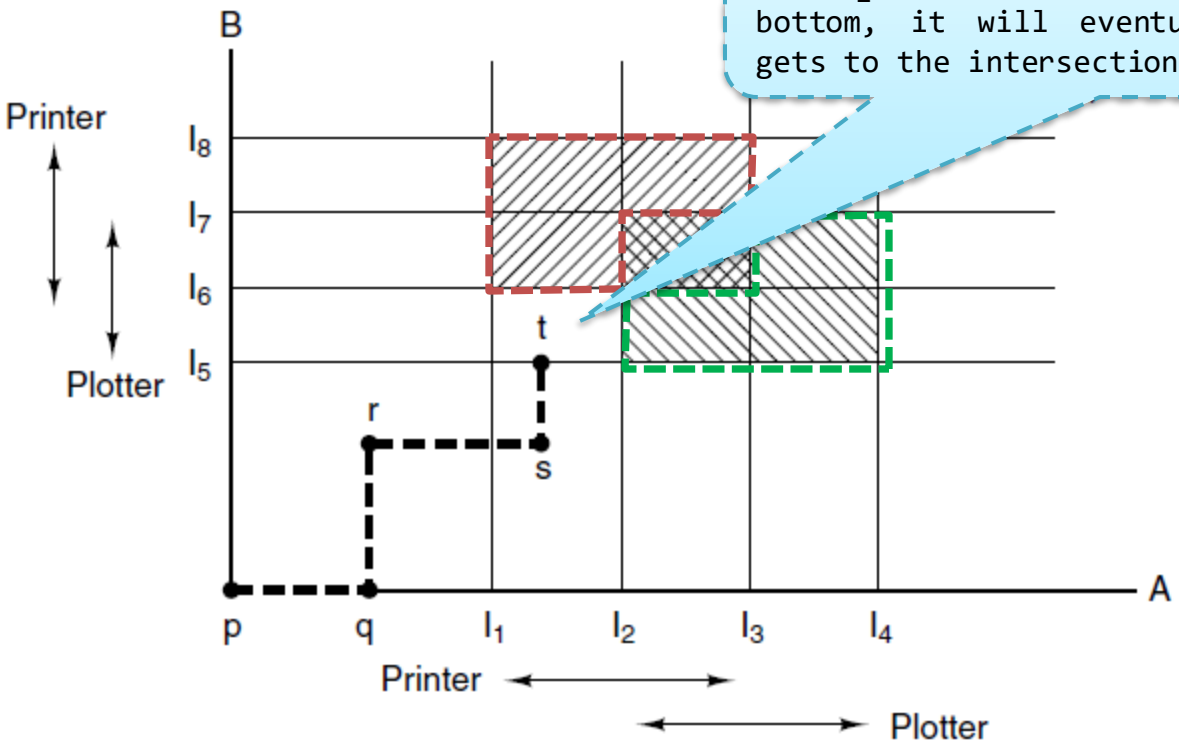
Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

===

DEADLOCK AVOIDANCE

Resource Trajectories

If the system ever enters the box bounded by I_1 and I_2 on the sides and I_5 and I_6 top and bottom, it will eventually deadlock when it gets to the intersection of I_2 and I_6.



Both processes have the printer
The mutual exclusion rule makes it impossible to enter this region

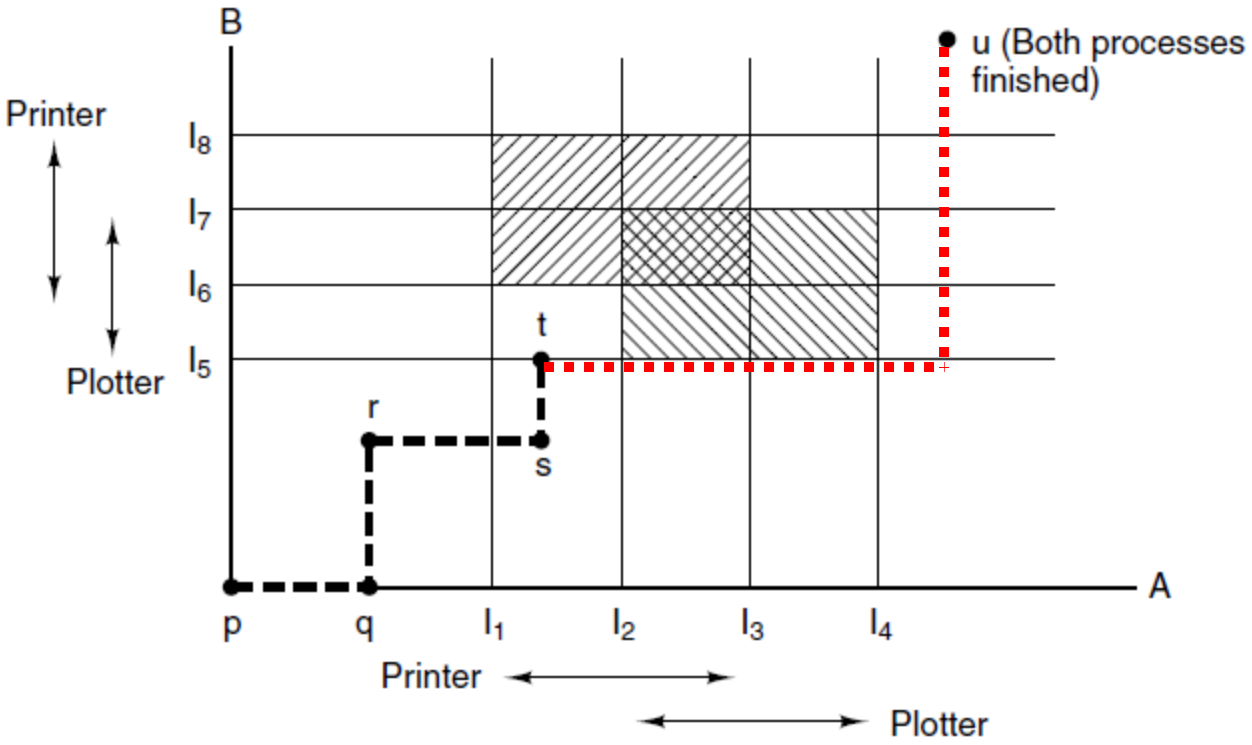
Both processes have the plotter
The mutual exclusion rule makes it impossible to enter this region

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

===

DEADLOCK AVOIDANCE

Resource Trajectories



At point t the only safe thing to do is run process A until it gets to I4. Beyond that, any trajectory to u will do.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Safe and Unsafe States

State:
E, A, C, and R matrices.

Safe state:
There is some scheduling order in which every process can run to completion even if all of them suddenly request their maximum number of resources immediately.

When a process requests a resource the system must decide if resource should be granted.

A safe state the system can guarantee that all processes will finish. From an unsafe state, no such guarantee can be given.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Safe and Unsafe States

Example :

A total of 10 instances of the resource exist, is (a) safe ?

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3
(a)

	Has	Max
A	3	9
B	4	4
C	2	7

Free: 1
(b)

	Has	Max
A	3	9
B	0	–
C	2	7

Free: 5
(c)

	Has	Max
A	3	9
B	0	–
C	7	7

Free: 0
(d)

	Has	Max
A	3	9
B	0	–
C	0	–

Free: 7
(e)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Safe and Unsafe States

Example :

A total of 10 instances of the resource exist, is (a) safe ?

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

9 - 3 = 6

4 - 2 = 2

7 - 2 = 5

Requests

Available resources(Free): 3 = 10 -(3+2+2)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Safe and Unsafe States

Example :

	Has	Max
✗	3	9
✓	2	4
✗	2	7

Free: 3
(a)

	Has	Max
✗	3	9
✓	4	4
✗	2	7

Free: 1
(b)

	Has	Max
✗	3	9
✓	0	–
✓	2	7

Free: 5
(c)

	Has	Max
✗	3	9
✓	0	–
✓	7	7

Free: 0
(d)

	Has	Max
✓	3	9
✓	0	–
✓	0	–

Free: 7
(e)

Demonstration that the state in (a) is safe.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Safe and Unsafe States

Example :

A total of 10 instances of the resource exist, is (b) safe ?

A requests and gets another resource

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max
A	4	9
B	2	4
C	2	7

Free: 2

(b)

	Has	Max
A	4	9
B	4	4
C	2	7

Free: 0

(c)

	Has	Max
A	4	9
B	—	—
C	2	7

Free: 4

(d)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Safe and Unsafe States

Example :

A total of 10 instances of the resource exist, is (b) safe ?

Available resources = 10 –(4 + 2 + 2) = 2

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max
A	4	9
	2	4
C	2	7

Free: 2

(b)

9 - 4 = 5

4 - 2 = 2

7 - 2 = 5

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

Safe and Unsafe States

Example :

A total of 10 instances of the resource exist, is (b) safe ?

Available resources = 10 –(4 + 2) = 4

	Has	Max
A	3	9
B	2	4
C	2	7

Free: 3

(a)

	Has	Max	
✗	4	9	9 - 4 = 5
✓	—	—	
✗	2	7	7 - 2 = 5

Free: 4

(b)

Demonstration that the state in (b) is not safe.

A’s request should not have been granted.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for a Single Resource

```
Customers : processes
Units     : resources
Banker    : the operating system
```

The banker's algorithm considers each request as it occurs, seeing whether granting it leads to a safe state. If it does, the request is granted; otherwise, it is postponed until later.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for a Single Resource

Example : Total resources = 10

	Has	Max
A	0	6
B	0	5
C	0	4
D	0	7

Free: 10

(a)

Safe

	Has	Max
A	1	6
B	1	5
C	2	4
D	4	7

Free: 2

(b)

Safe

	Has	Max
A	1	6
B	2	5
C	2	4
D	4	7

Free: 1

(c)

Unsafe

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for a Single Resource

Example : Total resources = 10 (part b)

	Has	Max	
✗ A	1	6	6-1=5
✗ B	1	5	5-1=4
✓ C	2	4	4-2=2
✗ D	4	7	7-4=3

Free: 2

	Has	Max	
✗ A	1	6	6-1=5
✗ B	1	5	5-1=4
✓ C	4	4	4-4=0
✗ D	4	7	7-4=3

Free: 0

	Has	Max	
✗ A	1	6	6-1=5
✗ B	1	5	5-1=4
✓ C	—	—	
✗ D	4	7	7-4=3

Free: 4

	Has	Max	
✗ A	1	6	6-1=5
✓ B	5	5	5-5=0
✓ C	—	—	
✗ D	4	7	7-4=3

Free: 0

	Has	Max	
✗ A	1	6	6-1=5
✓ B	—	—	
✓ C	—	—	
✗ D	4	7	7-4=3

Free: 5

	Has	Max	
✓ A	6	6	6-6=0
✓ B	—	—	
✓ C	—	—	
✗ D	4	7	7-4=3

Free: 0

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for a Single Resource

Example : Total resources = 10 (part b)

	Has	Max
A	—	—
B	—	—
C	—	—
D	4	7

Free: 6

7-4=3

	Has	Max
A	—	—
B	—	—
C	—	—
D	7	7

Free: 3

7-7=0

	Has	Max
A	—	—
B	—	—
C	—	—
D	7	7

Free: 3

	Has	Max
A	—	—
B	—	—
C	—	—
D	—	—

Free: 10

Safe

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for a Single Resource

Example : Total resources = 10 (part c)

	Has	Max	
A	1	6	6-1=5
B	1	5	5-1=4
C	2	4	4-2=2
D	4	7	7-4=3

Free: 2

	Has	Max	
A	1	6	6-1=5
B	2	5	5-2=3
C	2	4	4-2=2
D	4	7	7-4=3

Free: 1

B requests and gets another resource

Unsafe

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for Multiple Resources

1. Look for a row, R, whose unmet resource needs are <u>all smaller than or equal to A</u>. If no such row exists, system will eventually deadlock.

2. Assume the process of row chosen requests <u>all resources needed</u> and finishes. Mark that process as terminated, add its resources to the A vector.

3. Repeat steps 1 and 2 until either all processes are marked terminated (safe state) or no process is left whose resource needs can be met (deadlock)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for Multiple Resources

Example :

C: the current allocation matrix
R: the request matrix
E: the existing resources
P: the possessed resources
A: the available resources

E = P + A

Process	Tape drives	Plotters	Printers	CD ROMs
A	3	0	1	1
B	0	1	0	0
C	1	1	1	0
D	1	1	0	1
E	0	0	0	0

Resources assigned

C

Process	Tape drives	Plotters	Printers	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

R

E = (6342)
P = (5322)
A = (1020)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for Multiple Resources

Example :

Process	Tape drives	Plotters	Printers	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

R

A = [1 0 2 0]

A	1	1	0	0

A ≤ A ✗

A = [1 0 2 0]

B	0	1	1	2

B ≤ A ✗

A = [1 0 2 0]

C	3	1	0	0

C ≤ A ✗

A = [1 0 2 0]

D	0	0	1	0

D ≤ A ✓

A = [1 0 2 0]

E	2	1	1	0

E ≤ A ✗

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

The Banker's Algorithm for Multiple Resources

Example :

Process	Tape drives	Plotters	Printers	CD ROMs
A	1	1	0	0
B	0	1	1	2
C	3	1	0	0
D	0	0	1	0
E	2	1	1	0

Resources still needed

R

A = [1 0 2 0]

A	1	1	0	0

A ≤ A ✗

A = [1 0 2 0]

B	0	1	1	2

B ≤ A ✗

A = [1 0 2 0]

C	3	1	0	0

C ≤ A ✗

A = [1 0 2 0]

D	0	0	1	0

D ≤ A ✓

A = [1 0 2 0]

process D can finish, and then processes A or E, followed by the rest

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

- Resource Trajectories
- Safe and Unsafe States
- The Banker's Algorithm for a Single Resource
- The Banker's Algorithm for Multiple Resources

DEADLOCK PREVENTION

- Attacking the Mutual-Exclusion Condition
- Attacking the Hold-and-Wait Condition
- Attacking the No-Preemption Condition
- Attacking the Circular Wait Condition

OTHER ISSUES

- Two-Phase Locking
- Communication Deadlocks
- Livelock
- Starvation

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK PREVENTION

Attacking the Mutual-Exclusion Condition

Mutual exclusion

Each resource is either currently assigned to exactly one process or is available

If no resource were ever assigned exclusively to a single process, we would never have deadlocks, e.g., Data -> read only data

Allowing two processes to write on the printer at the same time will lead to chaos.

Solution: By spooling printer output, several processes can generate output at the same time.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK PREVENTION

Attacking the Hold-and-Wait Condition

Hold and wait

Processes currently holding resources that were granted earlier can request new resources

Require processes to request all their resources before starting execution. If everything is available, the process will be allocated whatever it needs and can run to completion.

If one or more resources are busy, nothing will be allocated and the process will just wait.

Problems:

- Many processes do not know how many resources they will need until they have started running.
- Resources will not be used optimally (ties up resources that other processes could be using)

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK PREVENTION

Attacking the Hold-and-Wait Condition

Hold and wait

Processes currently holding resources that were granted earlier can request new resources

A slightly different way to break the hold-and-wait condition is to require a process requesting a resource to first temporarily release all the resources it currently holds.

Then it tries to get everything it needs all at once.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK PREVENTION

Attacking the No-Preemption Condition

No preemption

Resources previously granted cannot be forcibly taken away from a process. They must be explicitly released by the process holding them.

This is not a very attractive option e.g., consider a process that is using a printer:

- Let process go halfway through its job.
- Then forcibly take away printer.
- Results can be unpredictable.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK PREVENTION

Attacking the Circular Wait Condition

Circular wait condition

There must be a circular list of two or more processes, each of which is waiting for a resource held by the next member of the chain.

- A process is entitled only to a single resource at any moment. If it needs a second one, it must release the first one.

  Problem : for a process that needs to copy a huge file from a tape to a printer, this restriction is unacceptable.

- Global numbering of all the resources. Processes can request resources whenever they want to, but all requests must be made in numerical order.
- The resulting resource graph is cycle-free.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK PREVENTION

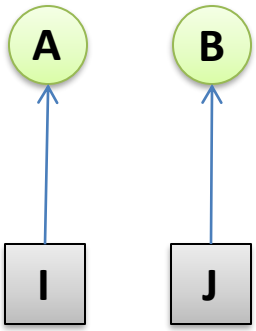
Attacking the Circular Wait Condition

Examples :

1. **Imagesetter**
2. **Printer**
3. **Plotter**
4. **Tape drive**
5. **CD-ROM drive**

A process may request first a printer(2) and then a tape drive(4), but it may not request first a plotter(4) and then a printer(2).

Deadlock
A requests resource j and B requests resource i



Either way, deadlock is impossible

(i<j)
A process (**B**) holding resource j cannot ask for resource i

(i>j)
A process (**A**) holding resource i cannot ask for resource j

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK PREVENTION

Summary of approaches to deadlock prevention

Assure that at least one of conditions is never satisfied

- Mutual exclusion
- Hold and wait
- No Preemption
- Circular wait

Condition	Approach
Mutual exclusion	Spool everything
Hold and wait	Request all resources initially
No preemption	Take resources away
Circular wait	Order resources numerically

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

DEADLOCK AVOIDANCE

- Resource Trajectories
- Safe and Unsafe States
- The Banker's Algorithm for a Single Resource
- The Banker's Algorithm for Multiple Resources

DEADLOCK PREVENTION

- Attacking the Mutual-Exclusion Condition
- Attacking the Hold-and-Wait Condition
- Attacking the No-Preemption Condition
- Attacking the Circular Wait Condition

OTHER ISSUES

- ~~Two-Phase Locking~~
- ~~Communication Deadlocks~~
- Livelock
- Starvation

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Livelock

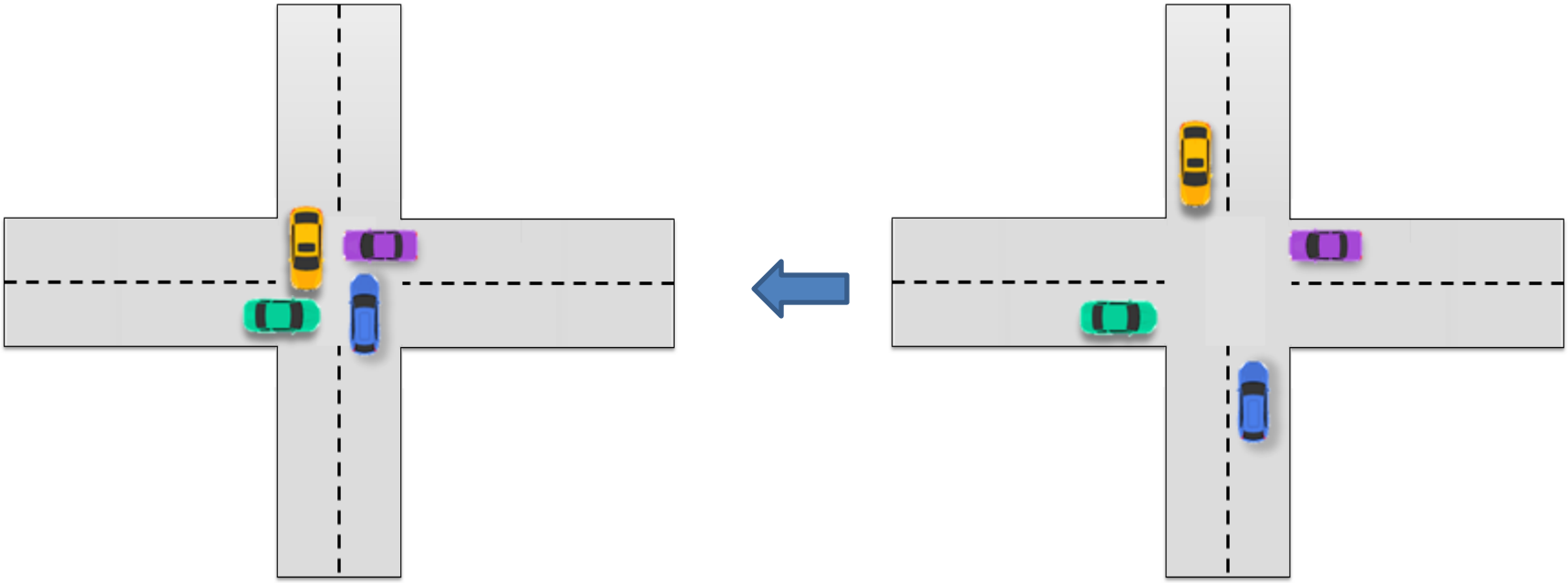
Two people trying to pass each other on the street when both of them politely step aside, and yet no progress is possible, because they keep stepping the same way at the same time.

Livelock is similar to deadlock in that <u>it can stop all forward progress, but it is technically different since it involves processes that are not actually blocked</u>.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Livelock

- The threads may look to be active and making progress,
- The overall system is not making meaningful progress towards a desired outcome.



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Livelock

Examples :

try_lock : the calling process tests a mutex and either grabs it or returns failure. In other words, it never blocks.

acquire_lock : also tries to grab the lock, but blocks if the lock is not available.

Now imagine a pair of processes running in parallel (perhaps on different cores) that use two resources.

```
void process_A(void) {
    acquire_lock(&resource_1);
    while (try_lock(&resource_2) == FAIL) {
        release_lock(&resource_1);
        wait_fixed_time();
        acquire_lock(&resource_1);
    }
    use_both_resources( );
    release_lock(&resource_2);
    release_lock(&resource_1);
}

void process_A(void) {
    acquire_lock(&resource_2);
    while (try_lock(&resource_1) == FAIL) {
        release_lock(&resource_2);
        wait_fixed_time();
        acquire_lock(&resource_2);
    }
    use_both_resources( );
    release_lock(&resource_1);
    release_lock(&resource_2);
}
```

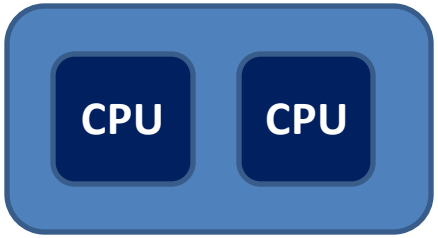
Polite processes that may cause livelock

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Livelock

Examples :




```
    void process_A(void) {
1 ->    acquire_lock(&resource_1);
2 ->    while (try_lock(&resource_2) == FAIL) {
3 ->        release_lock(&resource_1);
4 ->        wait_fixed_time();
5 ->        acquire_lock(&resource_1);
        }
        use_both_resources( );
        release_lock(&resource_2);
        release_lock(&resource_1);
    }
```

```
void process_A(void) {
    acquire_lock(&resource_2);
    while (try_lock(&resource_1) == FAIL) {
        release_lock(&resource_2);
        wait_fixed_time();
        acquire_lock(&resource_2);
    }
    use_both_resources( );
    release_lock(&resource_1);
    release_lock(&resource_2);
}
```

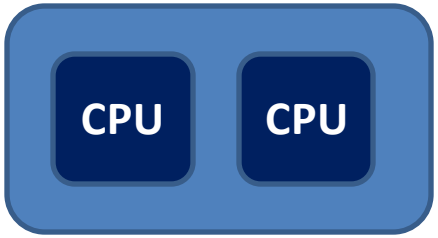
Polite processes that may cause livelock

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Livelock

Examples :




```
    void process_A(void) {
1 ->    acquire_lock(&resource_1);
2 ->    while (try_lock(&resource_2) == FAIL) {
3 ->        release_lock(&resource_1);
4 ->        wait_fixed_time();
5 ->        acquire_lock(&resource_1);
        }
        use_both_resources( );
        release_lock(&resource_2);
        release_lock(&resource_1);
    }
```

```
void process_A(void) {
    acquire_lock(&resource_2);
    while (try_lock(&resource_1) == FAIL) {
        release_lock(&resource_2);
        wait_fixed_time();
        acquire_lock(&resource_2);
    }
    use_both_resources( );
    release_lock(&resource_1);
    release_lock(&resource_2);
}
```

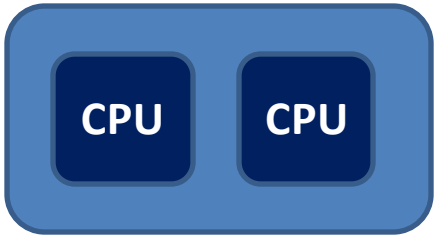
Polite processes that may cause livelock

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Livelock

Examples :




```
    void process_A(void) {
1 ->    acquire_lock(&resource_1);
2 ->    while (try_lock(&resource_2) == FAIL) {
3 ->        release_l esource_1);
4 ->        wait_fixec ();
5 ->        acquire_lock(&resource_1);
        }
        use_both_resources( );
        release_lock(&resource_2);
        release_lock(&resource_1);
    }
```

```
void process_A(void) {
    acquire_lock(&resource_2);
    while (try_lock(&resource_1) == FAIL) {
        release_ &resource_2);
        wait_fix ne();
        acquire_lock(&resource_2);
    }
    use_both_resources( );
    release_lock(&resource_1);
    release_lock(&resource_2);
}
```

Polite processes that may cause livelock

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Starvation

In a dynamic system, requests for resources happen all the time. Some policy is needed to make a decision about who gets which resource when. This policy, although seemingly reasonable, may lead to some processes never getting service even though they are not deadlocked, e.g.

1) priority scheduling in interactive systems.
2) SSF (Shortest Seek First in disk arm scheduling algorithms).

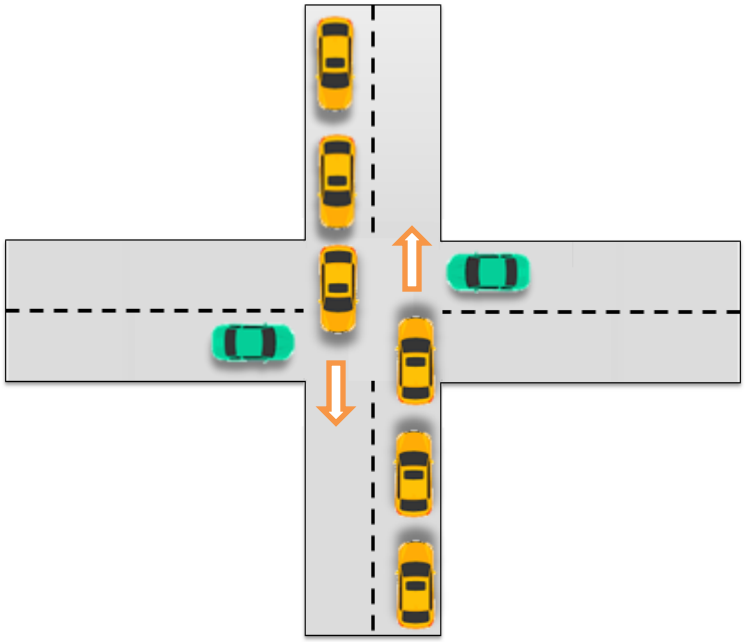
Starvation can be avoided by using a first-come, first-served resource allocation policy.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Starvation

If green cars yield the right of way to yellow cars, green cars cannot make any progress unless all yellow cars have passed.



Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

OTHER ISSUES

Starvation

Example:

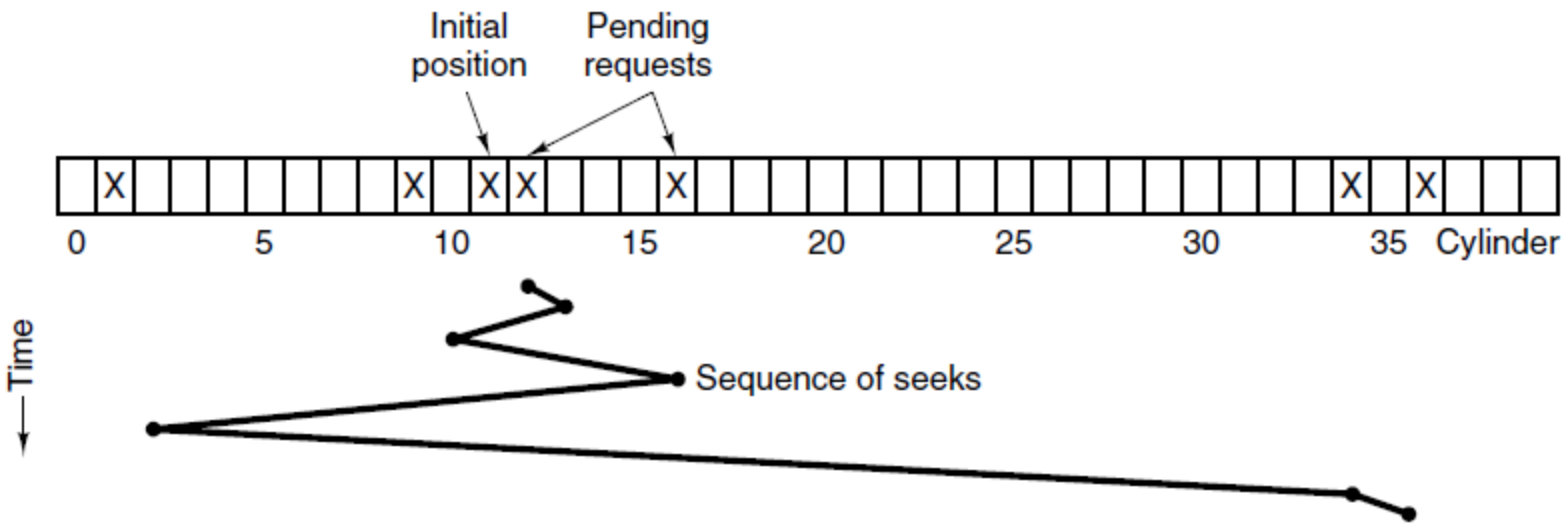
A request comes in to read a block on cylinder 11. While the seek to cylinder 11 is in progress, new requests come in for cylinders 1, 36, 16, 34, 9, and 12, in that order.

Tanenbaum & Bo, Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.

Disk Arm Scheduling Algorithms
SSF

Example:

11, 1, 36, 16, 34, 9, and 12 ⇒ **11, 12, 9, 16, 1, 34, and 36**



11 : (|11-12|=1) (|11-9|=2)(|11-16|=5)(|11-1|=10)(|11-34|=23)(|11-36|=25)
12 : (|12-9|=3) (|12-16|=4) (|12-1|=11) (|12-34|=22) (|12-36|=24)
9 : (|9-16|=7) (|9-1|=8)(|9-34|=25) (|9-36|=27)
16 : (|16-1|=15)(|16-34|=18)(|16-36|=20)
1 : (|1-34|=33) (|1-36|=35)
34 : (|36-34|=2)
36

Tanenbaum & Bo,Modern Operating Systems:4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved.